

基於 Slurm 與 Kubernetes 架構下 AI 伺服器 GPU 工作負載智慧排程技術之研究

鄭珽升(通訊作者)¹ 蕭友翰² 陳宗禧³

國立臺南大學資訊工程學系

{s11259043¹, s11259033²}@stumail.nutn.edu.tw, chents@mail.nutn.edu.tw³

摘要

本研究以一般大學實驗室共享 GPU 主機常見的資源閒置、GPU 利用率不足與批次工作管理困難為背景，探討如何在容器化環境中建立兼具彈性與穩定性的 AI 工作負載排程平台。旨在整合 Slurm 的批次排程能力與 Kubernetes 的容器編排及彈性擴縮能力，讓使用者能以熟悉流程提交工作，同時由系統自動完成資源啟動、分配與監控。本研究提出以 Slurm-on-Kubernetes 為核心的共享 GPU 排程架構，並導入 NVIDIA MPS 多行程共享與執行資源配額控制、監控介面與深度強化學習排程插件等策略，以支援多工作共享 GPU 及智慧化資源放置。在兩節點、每節點一張異質 GPU 的實機環境中，比較 FCFS、Backfill 與多種學習式策略於不同工作佇列深度下的表現。實驗結果顯示，學習式策略在平均工作完成時間上均顯著優於 FCFS 與 Backfill，且效益提升隨工作佇列深度加深而增加；然而在尾端延遲 P99 部分，FCFS 在中重度負載下表現最佳，顯示學習式策略在尾端延遲上仍有改進空間。

關鍵詞：GPU 資源排程、異質 GPU、Slurm、Kubernetes、深度強化學習。

Abstract

Shared GPU servers in university laboratories often suffer from resource underutilization, low GPU utilization, and difficulties in managing batch workloads. This study investigates how to build a flexible and reliable AI workload scheduling platform in a containerized environment. The proposed platform combines Slurm's batch scheduling capabilities with Kubernetes-based container orchestration and elastic scaling, allowing users to submit jobs through familiar workflows while the system automatically handles resource provisioning, allocation, and monitoring. We present a shared-GPU scheduling framework based on Slurm-on-Kubernetes. The framework incorporates NVIDIA Multi-Process Service (MPS) for concurrent multi-process GPU sharing and execution-resource quota control, a monitoring interface, and deep reinforcement learning scheduling policies for intelligent resource placement. The system was evaluated on a two-node testbed with one heterogeneous GPU per node across different job queue depths, comparing FCFS, Backfill, and various DRL policies (SAC, RDSAC, RLPD). Experimental results demonstrate that DRL-based policies consistently achieve lower average Job Completion Time (JCT) than FCFS and Backfill, with performance gains scaling significantly as the job queue deepens. However,

FCFS exhibits superior P99 tail latency under medium and heavy workloads, indicating that learning-based policies still have room for improvement in tail latency performance.

Keywords: GPU Resource Scheduling, Heterogeneous GPU, Slurm, Kubernetes, Deep Reinforcement Learning.

1. 緒論

1.1 簡介

Slurm Workload Manager 是高效能運算叢集最廣泛使用的工作排程系統之一，提供完整的工作提交、佇列管理、資源限額與 GPU 資源配置功能 [1]，其以「節點」為基礎的資源模型非常適合研究工作環境。然而 Slurm 以批次工作、優先權與節點資源管理為核心，容器生命週期與彈性擴縮通常需要額外的容器平台協調。Kubernetes 則是目前最主流的容器編排平台，能自動管理容器的部署、擴縮與健康監控 [2]，但其原生排程器 (kube-scheduler) 以服務導向設計為主，以尚未配置的 Pod 為單位，透過節點篩選與評分完成資源放置，未直接提供 Slurm 的批次佇列與 backfill 語意。因此有多項研究嘗試將 Slurm 與 Kubernetes 整合，以同時取得批次排程語意與雲端彈性管理能力；實作層面也有 Slinky [3] 等 Slurm-on-Kubernetes 方向的工具，顯示此類架構已具有實務需求與發展基礎。現有研究雖已分別探討 GPU 動態資源配置、Kubernetes 推論自動擴縮與共享 GPU 去碎片化 [4,5,6]，但不同方法通常各自聚焦於資源配置、服務副本或特定叢集模型，對真實 Slurm 提交流程中異質 GPU 與 MPS 配額的聯合排程，以及學習式策略失效時的安全回退，仍缺乏完整驗證。這形成本文的研究切入點：在保留 Slurm 批次工作語意的前提下，加入可部署的學習式決策層，並以真實 AI 工作負載檢驗其效能與穩定性。

1.2 研究貢獻

基於上述背景，本研究聚焦於三項問題。第一，在 MPS 配額與 GPU 型號差異同時存在時，聯合考慮工作選擇與 GPU 放置能否改善工作完成時間？第二，學習式策略接入真實 Slurm 提交流程後，能否在保有失效回退機制的條件下，與 FCFS 及 Backfill 進行公平比較？第三，在多種 workload 情境下，當學習式策略是否能進一步改善平均 JCT 與尾端 JCT？

本研究的核心貢獻在於將「異質 GPU + MPS + 真實 Slurm 流程」三者結合，主要貢獻有三。其一，

MPS 配額約束下的工作派遣與異質 GPU placement。不同於 UXP-RL [4] 主要決定 CPU-vs-GPU 資源類型、KIS-S [5] 調整 Kubernetes 推論副本數、DRR [6] 處理 GPU 碎片化，本研究的策略介面與模擬環境將「派哪個工作」與「放到哪張異質 GPU」建模為聯合離散動作。每個工作攜帶既定的 MPS fraction 需求作為 state 特徵與 action mask 的可行性約束，使決策在尊重 MPS 配額的前提下進行。此設計在模擬環境刻畫「job 選擇與異質 GPU placement 聯合決策」，實機則聚焦提交時 placement 相對於僅決定資源類型 [4] 或僅調整副本數 [5] 的可行性與行為差異。其二，失效安全的 Slurm 策略層整合。不同於 UXP-RL、KIS-S 等純模擬研究，本研究把學習式決策服務嵌入 Slurm job submission path，並提供 fail-safe 回退機制，當學習式排程服務逾時就自動回退至 Slurm 原生策略，在評估量測中未觀察到逾時，且策略服務失效時具備回退機制。此設計讓客製化 DRL 排程器得以在真實排程路徑中部署，而不需修改 Slurm 核心，這是先前純模擬研究 [4, 5] 未曾示範的部署可行性。其三，實機比較與效益分析。本研究於 RTX 4070 與 RTX 3080 異質環境，以 trace-derived 混合 AI 工作負載比較 FCFS、Backfill、SAC、RDSAC 與 RLPD，並使用統計檢定。結果顯示，學習式策略的工作完成時間優於 FCFS 與 Backfill，但是在尾端部分仍比 FCFS 略差。

2. 相關研究

2.1 GPU 叢集排程、分析與資源共享

近期研究聚焦於單節點內或單一叢集模型上的動態資源調度本身。Wang 等人的 DCUDA [7] 針對單節點多 GPU 情境，設計了一套輕量級核心／記憶體使用率監控機制，搭配近乎零開銷的執行中 CUDA 應用即時遷移，將 GPU 過載時間平均降低 78.3%、一般工作執行時間降低 42.1%。Sedighi 等人 [8] 則在 Alibaba 的 cluster-trace-gpu 生產工作負載軌跡之上，提出結合硬體與軟體分割的公平且需求感知動態資源配置演算法，於模擬環境中將 GPU 資源使用量降低達 88%。這兩項工作皆聚焦「資源配置本身如何隨工作負載動態調整」（即時遷移／再分割），評估分別侷限於單一多 GPU 節點與純模擬 trace 重放，並未涉及與 Slurm 的整合。

在 GPU 共享機制方面，NVIDIA MPS 允許多個 CUDA process 同時共享同一張 GPU 的運算資源 [9]，NVIDIA MIG 則在硬體層面將 GPU 切分成隔離的實例。學術上亦有更細緻的 partition 與時空共享研究，如多 GPU 伺服器上的時空共享服務 [10] 與現代 GPU 的階層式資源分割 [11]。針對深度學習工作負載，也有專門的叢集排程系統：Gandiva [12] 以 introspective 排程與工作遷移提升利用率，Tiresias [13] 以近似 age-based 的優先序縮短 JCT，

MLaaS 生產叢集分析 [14] 則刻畫了大規模異質 GPU 叢集的工作特性。這些方法多假設整卡分配或同質 GPU，較少把 MPS fraction 作為排程流程的決策脈絡。

2.2 強化學習排程

Lin 等人 [4] 提出 UXP-RL：一個以 DQN 為核心、涵蓋前處理／訓練／推論三類任務、可部署為集中式或分散式排程器、並跨雲／邊／霧三層架構運作的 CPU-GPU 任務排程演算法。其於模擬環境中，集中式排程器將平均週轉時間相較 SJF／FCFS 與 TYPE 啟發式（依 GPU 需求高低分類任務）分別降低 57.81%、57.28% 與 27.66%；分散式排程器則因能將長訓練工作卸載至雲端而釋放邊／霧資源，把推論任務週轉時間相較集中式再降低 89.07%。同年 Zhang 等人的 KIS-S [5] 以 PPO 訓練一個 GPU-aware 的 Kubernetes 推論自動擴縮策略 (KIScaler)，完全於自建模擬器 (KISim) 中訓練後即零樣本部署，於多種流量情境下平均獎勵提升 75.2%、p95 延遲相較 CPU 基準降低最多 6.7 倍；其問題設定是調整副本數的自動擴縮，而非本研究的工作放置排程。Wu 等人的 DRR [6] 則針對 GPU 共享叢集因分享機制、工作異質性與非同步生命週期造成的碎片化問題，以模仿學習從既有啟發式暖啟動一個深度強化學習去碎片化排程器，並輔以多尺度策略最佳化平衡探索與利用；其同時於實體 Kubernetes 測試床與大規模模擬叢集上驗證，平均碎片率降低 50%，是本節所列中少數同時涉及真實 Kubernetes 部署的學習式排程器。

2.3 異質與邊緣 GPU 排程

Tsenos 與 Kalogeraki [15] 針對缺乏原生虛擬化支援的邊緣 GPU 提出一套硬體無關的時空共享機制：為每個行程建立 cgroup、動態調整其 duty cycle 來實現優先權式與截止期限式排程。Majeed 等人 [16] 則以系統性文獻回顧整理 NVIDIA Jetson 系列邊緣 SoC 上的 DNN 排程器，區分規則式與最佳化式兩大類，並整理其記憶體競爭、跨加速器轉移成本與靜動態排程的權衡；其排程粒度是單一 DNN 模型內的層級。這些邊緣場景的資源與延遲約束與資料中心叢集不同，惟顯示異質硬體上的細粒度排程是一個活躍的研究方向。此外，異質 k8s 叢集上的 RL 排程 [17]、混合學習與最佳化排程 [18] 與語意感知的 LLM 叢集排程 [19] 亦顯示學習式方法在異質資源分配上的潛力。

3. 研究目的與系統架構

3.1 研究目的

本研究的目標是在異質 GPU 與 NVIDIA MPS 環境中，學習一個能在 MPS 配額約束下決定工作派遣與分配到指定 GPU 的排程策略，以降低平均 JCT，並探討在追求整體排程效率時，如何透過風險敏感(Risk-sensitive)策略來控制尾端 JCT 的劣化。更具體而言，本研究要回答兩個問題：第一，DRL 是否能在模擬環境中學到比 Slurm 內建排程更好的 GPU + MPS 排程策略；第二，此學習式策略部署到真實異質 GPU 叢集後，能否勝過 FCFS、Backfill 內建策略。

3.2 系統架構

見下圖 1，學習式策略以服務形式接入 Slurm 的工作提交流程：策略讀取工作、GPU、MPS 剩餘容量與佇列狀態等，輸出建議的工作與 GPU placement，並由系統於提交時據以綁定節點。若策略服務逾時或回傳不可行決策，系統自動回退至 Slm 原生路徑，確保排程核心不被阻塞。記憶體用量、JCT 等資訊，將資料收集起來以供後續訓練或 RLPD (Reinforcement Learning with Prior Data) [20] 微調使用。

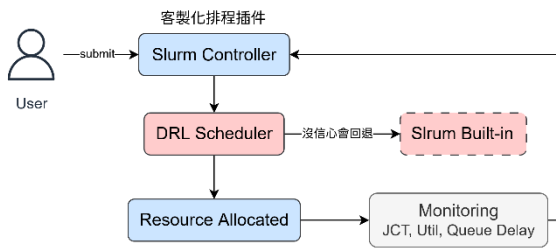


圖 1 工作提交到排程決策流程

本研究使用 Kubernetes (k3s) 部署 Slurm controller、worker、自訂排程器、監控服務與相關容器 [2]，底層 GPU 基礎建設是透過 Kubernetes Dynamic Resource Allocation (DRA) [21] 宣告與取得裝置，工作層級的 MPS 配額仍由 Slurm 與對應的 MPS 執行環境落實。Kubernetes 在本文中不負責排程決策，它只提供容器化部署、服務健康檢查、網路與生命週期管理。此方向與將 Slurm 整合進 Kubernetes 的 Slinky [3] 互補，Slinky 提供 Slurm-on-Kubernetes 部署基礎，本研究則在 Slurm 排程路徑上加入具失效回退機制的學習式策略層。

4. 排程技術

4.1 Slurm 內建排程演算法

以 Slurm 原生排程能力作為穩定基線，而非重寫排程核心。例如 backfill 允許在不延後高優先權工作的前提下，讓資源需求較小、執行時間較短的工作提前插隊執行，緩解大工作長期佔用資源造成的閒置，此外也納入更保守的 FCFS 作為基準對照，用以檢驗學習式策略相對 Slurm 開箱即用能

力是否確有改善。

4.2 深度強化學習策略

在強化學習設計上，每次有工作可排程時，代理人觀察叢集狀態來選擇一個工作與其 GPU 放置，並在工作完成後根據 JCT 取得 reward。狀態包含四類資訊：

- Job features：工作類型、預估 runtime、VRAM 需求、MPS 需求與等待時間。
- GPU features：GPU 型號、GPU 利用率、SM 利用率、記憶體用量、可用 MPS 資源、目前共置工作數。
- Queue features：佇列長度、前 K 個工作的需求、等待時間分布與到達率。
- History features：近期完成工作 JCT、slow-down 與各 GPU 的負載變化。

動作是「選擇佇列前 K 個工作中的哪一個」與「將其放置到哪張 GPU」的聯合離散決策，並且有暫不派遣的選項。在實機評估中，僅工作選擇（順序）透過 Slurm Priority 調整，placement 交由 Slurm 自己執行。若某放置的 GPU 剩餘 MPS 資源不足以容納該工作的請求，動作遮罩會在推論前屏蔽該選項，避免產生不可執行決策。在本研究的 2 GPU 實驗平台中， $K=16$ ，動作空間總共有 33 個離散動作，observation 維度為 168。強化學習的 Reward 的設計目標是降低使用者感受到的等待與完成時間，故採以 JCT 為核心的形式：

$$Reward = \frac{-JCT}{20000} - 5 \left(\frac{JCT}{20000} \right)^2 + [\gamma \phi(s') - \phi(s)] \quad (1)$$

$$\phi(s) = \frac{\sum_{\text{pending job}} \text{wait_time}}{20000N} - 5 \frac{\text{std}(\text{每節點閒置MPS})}{\text{每節點的總MPS}} \quad (2)$$

其中 JCT 代表從工作提交到完成的花費時間，第二項可以避免尾端太大，等待與負載均衡兩項以 potential-based shaping 整合成第三項。此 reward 設計讓模型不只最佳化單一工作，而是學習長期排程結果。

本研究比較以下 DRL 方法：

- Discrete SAC：將 Soft Actor-Critic 延伸到離散動作空間，以 categorical 策略取代高斯策略、以期望估計熵項，適合 job 選擇與 GPU 分配這類有限離散動作 [22]。
- RDSAC-mean：根據 X. Ma et al. 提出 Distributional Soft Actor-Critic for Risk-Sensitive Reinforcement Learning（以下簡稱 RDSAC）。分布式 critic 建模回報分布，但 actor 主要依平均回報決策 [23, 24]。

- RDSAC-cvar：在 RDSAC 上加入 CVaR 風險敏感目標，使策略更重視尾端 JCT。
- RLPD：以真實資料對模擬訓練出的模型進行微調，來縮小模擬到真實的差距。

5. 實驗與評估方法

5.1 實驗環境與訓練

本研究使用一個小規模異質 GPU 實機叢集進行部署與評估，相關環境如表 1 所示。訓練資料集使用 Alibaba GPU Trace [25] 與 Microsoft Philly Trace [26]，前者用於參考生產 MLaaS 工作的到達率、工作長度與資源需求分布[14]，後者用於參考多租戶 GPU training workload 的佇列與 JCT 特性[27]。同時在本研究叢集上執行 BERT 推論、ResNet 訓練、Qwen 微調與 cuBLAS 矩陣運算等真實 AI 工作。

表 1 實驗環境列表

項目	設定
Node 1	Intel Core i7-10700、64 GB RAM、NVIDIA RTX 4070
Node 2	Intel Core i7-9700、8 GB RAM、NVIDIA RTX 3080
作業系統	Ubuntu 24.04.4 LTS (kernel 7.0.0)
NVIDIA driver / CUDA	580.167.08 / CUDA 13.0
部署平台	k3s v1.34.6、containerd 2.2.2
GPU 資源宣告	Kubernetes Dynamic Resource Allocation (DRA) driver v0.4.1
排程器	Slurm 23.11.7 with GRES/TRES and MPS
GPU sharing	NVIDIA MPS，MPS 可用配額為 25%、50%、75%、100%
網路	同一區域網路，節點間 RTT $\approx$ 0.16 ms
監控	SM 利用率、記憶體用量、job event、佇列延遲

直接在實際環境從頭訓練的話，強化學習需要數十萬到數百萬個 transition，真實叢集一個編排決策對應一個跑數分鐘至數小時的任務，湊滿樣本要等數月，因此使用兩段式做法。首先在模擬環境大量訓練，產出基本模型。接著上線部署，記錄真實叢集 (observation, action, reward) 資料。最後經過 RLPD 用真實資料把基本模型 RDSAC-cvar 微調策略，目前使用約 2789 筆真實叢集工作紀錄，此微調資料是使用不同時間段的工作，不包含評估資料。深度強化學習的訓練參數如表 2 所示：

表 2 訓練參數設定

項目	數值
優化器 / 學習率	Adam / 3×10^{-4}
隱藏層 (MLP trunk)	(256, 256) + LayerNorm
batch size / UTD ratio	256 / 4
n-step return	10
風險目標 / tail mass β	CVaR / 0.25 (RDSAC-cvar 特有)
溫度 α	固定為 0.05
Reward	如式(1)
訓練長度	1×10^5 curriculum env-steps

5.2 實機評估

在 BERT 推論、ResNet 訓練、Qwen 微調與 cuBLAS 矩陣運算混合工作負載進行評估，數量占比分別為 30%、30%、30% 和 10%，以一個評估工作隨機生成器，共 10 個亂數種子，每個亂數種子有 150 個工作，工作採 Poisson 到達，相鄰工作間隔服從指數分布，並觀察較慢到較高的工作提交頻率，進而評估 DRL 在不同深度的 job queue 的效益提升。評估結果由各亂數種子先計算統計量，再對 16 個亂數種子取未加權平均與標準差得到 JCT，並以 Backfill 為基準用 Wilcoxon 檢定顯著性，結果如表 3、4、5。

表 3 輕度提交頻率的評估結果

排程法	JCT (s)	P99 (s)	檢定 p
FCFS	167.5 $\pm$ 27.2	306.1 $\pm$ 43.0	0.010
Backfill	151.8 $\pm$ 31.3	652.5 $\pm$ 70.0	-
SAC	150.1$\pm$30.7	315.6 $\pm$ 101.2	0.85
RDSAC -mean	151.1 $\pm$ 33.9	325.2 $\pm$ 104.4	0.92
RDSAC -cvar	151.2 $\pm$ 33.3	334.8 $\pm$ 143.1	1.00
RLPD	<u>150.5$\pm$34.7</u>	302.4$\pm$71.6	0.77

表 4 中度提交頻率的評估結果

排程法	JCT (s)	P99 (s)	檢定 p
FCFS	286.5 $\pm$ 20.6	546.1$\pm$30.8	0.004
Backfill	260.7 $\pm$ 19.3	750.8 $\pm$ 31.1	-
SAC	233.8 $\pm$ 15.8	<u>605.3$\pm$93.0</u>	0.002
RDSAC -mean	235.7 $\pm$ 17.9	628.8 $\pm$ 64.5	0.002
RDSAC -cvar	233.0$\pm$15.7	642.2 $\pm$ 72.3	0.002
RLPD	<u>233.2$\pm$18.3</u>	613.9 $\pm$ 77.1	0.002

表 5 重度提交頻率的評估結果

排程法	JCT (s)	P99 (s)	檢定 p
FCFS	314.3 $\pm$ 20.9	604.3$\pm$30.5	0.19
Backfill	304.6 $\pm$ 18.4	767.8 $\pm$ 32.1	-
SAC	265.4$\pm$22.1	<u>667.1$\pm$76.5</u>	0.002

RDSAC -mean	270.0±15.5	679.0±64.0	0.002
RDSAC -cvar	<u>265.6±15.6</u>	677.9±74.3	0.002
RLPD	269.6±14.6	684.2±58.1	0.002

在輕度負載下評估時，所有學習式策略的平均 JCT 皆顯著勝過 FCFS，且與 Backfill 差距在 0.2~0.9%，但在 P99 上 RLPD 為最佳，其次是 FCFS 及 SAC。在中度負載下評估時，所有學習式策略的平均 JCT 皆顯著勝過 Backfill 約 9.4~10.5%，表現最佳的策略是 SAC，但是在 P99 上 FCFS 為最佳，其次是 SAC 及 RLPD。在深負載下評估時，所有學習式策略的平均 JCT 皆顯著勝過 Backfill 約 11.3~13.0%，表現最佳的策略是 SAC，但是在 P99 上 FCFS 為最佳，其次是 SAC 及 RDSAC-cvar。

以下分析不同工作提交頻率來評估 DRL 在不同深度的 job queue 相較於 Backfill 的效益提升程度，結果如表 6 所示，其中定義結果如式(3)，正值表示比 backfill 好：

$$\Delta JCT\% = 100 \times \frac{JCT_{backfill} - JCT_{arm}}{JCT_{backfill}} \quad (3)$$

表 6 各策略於不同工作佇列深度的效益 (%)

排程法	輕度	中度	重度
FCFS	-11.7±10.0	-10.0±4.5	-3.3±6.6
Backfill	-	-	-
SAC	0.7±8.2	10.2±4.4	13.0±3.1
RDSAC -mean	0.2±10.9	9.4±5.0	11.3±1.9
RDSAC -cvar	0.2±10.5	10.5±3.0	12.7±3.4
RLPD	0.9±10.3	10.4±5.2	11.4±4.0

5.3 系統行為量測

除排程品質外，本研究在評估時亦量測學習式決策路徑的系統行為，以檢驗其失效安全整合是否非侵入。依線上服務的信心值將每次決策分類為 RL 主導、低信心回退，或暫不派遣，結果如表 7。可以發現決策延遲為次毫秒級，且未觀察到超過 150 ms 門檻的決策，逾時型回退不會因決策過慢而觸發；實際 placement 中僅 12% 因低信心回退至 Slurm 原生排程。

表 7 排程器決策行為

指標	數值
決策延遲時間(ms) (mean/p50/p95/p99/max)	0.19/0.18/0.26/0.27/7.2
逾時(>150 ms)	0.00% (0 / 76,099)
低信心回退比例	12.0% (115 / 959)
RL 主導放置比例	88.0% (844 / 959)

6. 結論與未來展望

6.1 結論

本研究實作了可在異質 GPU 與 NVIDIA MPS 配額約束下輸出工作選擇與 GPU 放置的深度強化學習策略，並透過 Slurm job submission path 整合到真實排程流程。相較傳統只選 GPU 或只做固定規則的方法，本研究在排程框架中整合了 GPU 型號差異、MPS 配額、工作特徵、佇列狀態與回饋訊號，並以失效安全設計確保排程核心穩定。

實驗結果顯示，SAC、RDSAC、RLPD 在平均工作完成時間上優於 FCFS 及 Backfill，且效益隨工作佇列深度提升，但是在 P99 部分 FCFS 為最佳，表示學習式策略在尾端仍有待改進的空間。惟本研究在實機評估是小規模實驗室環境，在較大型的叢集的效能有待驗證。

6.2 未來展望

- (1) 擴展至更大、更高競爭的叢集：擴展至更多節點與 GPU，檢驗學習式策略的效益是否隨叢集規模與競爭程度進一步增強。
- (2) MIG + MPS 配額混合 partition：同時納入硬體級隔離與軟體級共享，建立更完整的 GPU sharing 動作空間。
- (3) Energy-aware scheduling：將功耗、能效與碳排納入 reward，使排程器兼顧效能與能源效率的最佳化。

參考文獻

- [1] SchedMD, "Slurm workload manager," <https://slurm.schedmd.com>, 2024.
- [2] Kubernetes Authors, "Kubernetes," <https://kubernetes.io>, 2024.
- [3] SchedMD, "Slink: Slurm in Kubernetes," <https://slinky.schedmd.com>, 2024.
- [4] Y.-D. Lin, Y.-T. Ling, Y.-C. Lai, and D. Sudyana, "Reinforcement learning for AI as a service: CPU-GPU task scheduling for preprocessing, training, and inference tasks," *IEEE Transactions on Network and Service Management*, vol. 22, no. 4, 2025.
- [5] G. Zhang, W. Guo, Z. Tan, Q. Guan, and H. Jiang, "KIS-S: a GPU-aware Kubernetes inference simulator with RL-based auto-scaling," in *IEEE IPCCC*, pp. 1–8, 2025, doi: 10.1109/IPCCC66453.2025.11304654.
- [6] Q. Wu, P. Chen, and Y. Wang, "Defragmentation scheduling with deep reinforcement learning in shared GPU clusters," in *ACM SoCC*, pp. 402–415, 2025, doi: 10.1145/3772052.3772242.
- [7] X. Wang, Y. Li, F. Guo, Y. Xu, and J. C. S. Lui, "Dynamic GPU scheduling with multi-resource awareness and live migration support," *IEEE Transactions on Cloud Computing*, vol. 11, no. 3, pp. 3153–3167, 2023, doi: 10.1109/TCC.2023.3264242.
- [8] H. Sedighi, F. Wuhib, and R. H. Glioth, "Dynamic task scheduling and adaptive GPU resource allocation in the cloud," *IEEE Transactions on Network and Service Management*, vol. 23, 2026.
- [9] NVIDIA Corporation, "Multi-Process service (MPS)," NVIDIA Documentation, 2024.
- [10] S. Choi, S. Lee, Y. Kim, J. Park, Y. Kwon, and J. Huh, "Serving heterogeneous machine learning models on multi-GPU servers with spatio-temporal sharing," in *USENIX ATC*, 2022.

- [11] U. Saroliya, E. Arima, D. Liu, and M. Schulz, "Hierarchical resource partitioning on modern GPUs: a reinforcement learning approach," in *IEEE CLUSTER*, pp. 185–196, 2023, doi: 10.1109/CLUSTER52292.2023.00023.
- [12] W. Xiao, R. Bhardwaj, R. Ramjee, et al., "Gandiva: introspective cluster scheduling for deep learning," in *USENIX OSDI*, 2018.
- [13] J. Gu, M. Chowdhury, K. G. Shin, et al., "Tiresias: a GPU cluster manager for distributed deep learning," in *USENIX NSDI*, 2019.
- [14] Q. Weng, W. Xiao, Y. Yu, et al., "MLaaS in the wild: workload analysis and scheduling in large-scale heterogeneous GPU clusters," in *USENIX NSDI*, 2022.
- [15] M. Tsenos and V. Kalogeraki, "Exploring GPU-based workload scheduling techniques for edge computing," in *IEEE IC2E*, 2025.
- [16] A. A. Majeed, M. Meribout, and S. M. Sali, "Scheduling techniques of AI models on modern heterogeneous edge GPU: a critical review," *IEEE Transactions on Industrial Informatics*, vol. 22, no. 4, 2026.
- [17] S. Dong, B. Zheng, L. Pan, and S. Liu, "A reinforcement learning-based approach for scheduling machine learning training tasks in heterogeneous Kubernetes clusters," *Future Generation Computer Systems*, vol. 182, art. 108459, 2026.
- [18] S. Dongare, R. I. S. Khan, H. Albahar, N. Zhao, D. Melendez Maita, and A. R. Butt, "Hybrid learning and optimization-based dynamic scheduling for DL workloads on heterogeneous GPU clusters," in *ACM SoCC*, 2025.
- [19] Z. Wang, Q. Hu, A. Klimovic, T. Zhang, Y. Wen, P. Sun, and D. Lin, "Semantic-aware scheduling for GPU clusters with large language models," *arXiv:2510.03334*, 2025 (unpublished).
- [20] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine, "Efficient online reinforcement learning with offline data (RLPD)," in *ICML*, 2023.
- [21] Kubernetes Authors, "Dynamic resource allocation (DRA)," *Kubernetes Documentation*, 2025.
- [22] P. Christodoulou, "Soft actor-critic for discrete action settings," *arXiv:1910.07207*, 2019 (unpublished).
- [23] W. Dabney, G. Ostrovski, D. Silver, and R. Munos, "Implicit quantile networks for distributional reinforcement learning," in *ICML*, 2018.
- [24] X. Ma, J. Chen, L. Xia, J. Yang, Q. Zhao, and Z. Zhou, "DSAC: distributional soft actor-critic for risk-sensitive reinforcement learning," *Journal of Artificial Intelligence Research*, vol. 83, 2025.
- [25] Alibaba, "Cluster data: production cluster traces," *GitHub repository*. [Online]. Available: <https://github.com/alibaba/clusterdata>. Accessed: Jun. 25, 2026.
- [26] Microsoft Research, "Philly traces," *GitHub repository*. [Online]. Available: <https://github.com/msr-fiddle/philly-traces>. Accessed: Jun. 25, 2026.
- [27] M. Jeon, S. Venkataraman, A. Phanishayee, et al., "Analysis of large-scale multi-tenant GPU clusters for DNN training workloads," in *USENIX ATC*, 2019.